Performance Engineering con k6

Requisitos

1. Descomprima el ZIP y abra PowerShell dentro de la carpeta o clone el repo.

2. Inicie Docker Desktop.

3. Levante la aplicación con: docker compose up -d --build performance-app

4. Verifique en el navegador: http://localhost:3000

5. Compruebe k6 con: docker compose --profile tools run --rm k6 version

6. Antes de una prueba importante puede reiniciar el estado con POST http://localhost:3000/api/admin/reset.

Regla de trabajo

Cada reto debe incluir objetivo, tipo de carga, checks, thresholds/SLO, ejecución, métricas observadas y una conclusión técnica. No basta con obtener una salida de k6.

Reto 1 - Correlación y autenticación

- Complete 01-auth-correlation.js.
- Obtenga un token con POST /api/auth/login y reutilícelo en GET /api/products/1.
- Valide status, JSON y X-Cache.
- Cree al menos un threshold por endpoint.
- Explique por qué la correlación de token es necesaria en una prueba real.

Reto 2 - Medición de caché

- Complete 02-cache-analysis.js.
- Consulte el mismo producto al menos 5 veces.
- Distinga MISS y HIT usando el header X-Cache.
- Calcule el porcentaje de mejora entre origen y caché.
- Defina un SLO específico para respuestas HIT.

Reto 3 - Modelo de carga mixto

- Complete 03-mixed-workload.js usando scenarios.
- Implemente al menos: ramping-vus, constant-arrival-rate y constant-vus.
- Modele aproximadamente 60% navegación, 30% búsqueda y 10% órdenes.
- Use tags para separar métricas.
- Defina thresholds diferentes por endpoint.
- Explique por qué arrival rate es distinto de VUs.

Reto 4 - Rate limiting

- Complete 04-rate-limit.js.
- Genere suficiente tráfico para provocar HTTP 429.
- Mida el porcentaje de peticiones limitadas.
- Inspeccione Retry-After.

- Diseñe un threshold que falle si 429 supera 3%.
- Determine si el problema es capacidad, protección o un diseño incorrecto de la prueba.

Reto 5 - Idempotencia bajo carga

- Complete 05-idempotency.js.
- Envíe dos veces una orden con la misma Idempotency-Key.
- Compruebe que no se cree una segunda orden.
- Repita con claves diferentes.
- Cree una métrica personalizada para duplicados.
- Explique qué riesgo de negocio evita la idempotencia.

Reto 6 - Flujo asíncrono y correlación de jobId

- Complete 06-async-export.js.
- POST /api/exports devuelve 202 + jobId.
- Realice polling hasta que GET /api/exports/:jobId devuelva completed.
- Mida tiempo end-to-end con Trend.
- El SLO debe aplicarse al flujo completo, no solo a una petición HTTP.
- Evite polling infinito: establezca límite de intentos.

Reto 7 - Falla de dependencia

- Complete 07-dependency-failure.js.
- Genere carga sobre /api/payments.
- Mida HTTP 502 mediante una Rate personalizada.
- Valide un SLO de error menor al 4%.
- Compare http_req_failed con su métrica de negocio.
- Explique si el sistema debería hacer retry automáticamente y qué riesgo tendría.

Reto 8 - Encontrar el breaking point

- Complete 08-breaking-point.js.
- Use ramping-arrival-rate o una estrategia equivalente.
- Incremente la carga hasta observar degradación evidente.
- Pruebe simultáneamente search y report.
- Defina SLOs ANTES de ejecutar.
- Registre el nivel de carga donde aparece 503 o p95 deja de cumplir.
- Consulte /api/admin/stats y relacione maxInFlight con el resultado.

Reto 9 - Spike y recuperación

- Complete 09-spike-recovery.js.
- Diseñe una carga normal, un pico abrupto y un periodo posterior de recuperación.
- Compare p95 durante y después del pico.
- Determine si el sistema realmente vuelve a su nivel normal.
- Diferencie resiliencia de capacidad máxima.

Reto 10 - Release Gate

- Complete 10-release-gate.js.
- Incluya login, producto, búsqueda, orden, pago y export asíncrono.

- SLO: error global <2%, checks >98%, producto p95<450ms, búsqueda p95<700ms, órdenes p95<900ms, payment_failures<4%, export E2E p95<3000ms.
- Implemente handleSummary() y escriba /results/summary.json.
- Decida GO o NO-GO con base en evidencia.
- Si falla, identifique exactamente qué SLO impide liberar.

Comandos de ejecución

Ejecute cada archivo con el patrón:

```
docker compose --profile tools run --rm k6 run /scripts/01-auth-correlation.js
```

Cambie el nombre del archivo según el reto.

Registro final de resultados

Reto	Carga	p95	p99	Error %	Req/s	PASS/FAIL
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						

Conclusión técnica

1. ¿Cuál fue el primer síntoma de degradación?

2. ¿Cuál fue el breaking point aproximado?

3. ¿Qué endpoint tuvo el peor comportamiento y por qué?

4. ¿Qué diferencia observó entre concurrencia (VUs) y tasa de llegada (arrival rate)?

5. ¿Qué impacto tuvo el caché?

6. ¿Qué porcentaje real de fallas presentó la dependencia de pagos?

7. ¿El rate limiting fue un fallo o un mecanismo de protección?

8. ¿El sistema se recuperó después del spike?

9. Decisión final: GO o NO-GO. Justifique con métricas.